Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Ingeniería
Carrera de Ingeniería de Software

Asignatura: Sistemas Distribuidos
Actividad: TA-IND-01 — Informe Académico Individual

Estudiante: Luis Alberto Paca Pilataza
Curso/Paralelo: 6 A
Fecha: 13 de noviembre de 2026

Repositorio GitHub:

https://github.com/Luispaca2002/TA-IND-01-Informe-Acad-mico-Individual-An-lisis-Cr-tico-de-Sistemas-Distribuidos-Unidad-1-

Resumen

Este documento presenta un análisis crítico de la arquitectura distribuida de MongoDB, enfocado específicamente en su implementación de Replica Sets para garantizar alta disponibilidad y tolerancia a fallos. Se examina el sistema a través de los conceptos teóricos fundamentales, incluyendo las transparencias de distribución, el modelo de comunicación, y los compromisos arquitectónicos establecidos por los teoremas CAP y PACELC. A diferencia de un análisis puramente documental, este informe se sustenta en la ejecución de un entorno local distribuido mediante contenedores Docker, donde se provocó deliberadamente la caída del nodo primario para observar, medir y documentar el proceso algorítmico de elección de líder y recuperación del clúster. Las conclusiones obtenidas confirman que MongoDB prioriza la consistencia frente a las particiones de red (CP), sacrificando disponibilidad temporal durante los tiempos de elección, lo cual define su idoneidad para entornos donde la integridad transaccional supera la necesidad de latencia absoluta.

1. Introducción

En el panorama actual de la gestión de datos a gran escala, las bases de datos relacionales monolíticas presentan cuellos de botella severos al intentar escalar horizontalmente. MongoDB se diseñó para mitigar este problema mediante un modelo orientado a documentos (NoSQL), permitiendo distribuir masivamente la información en múltiples servidores sin imponer esquemas rígidos en disco.

Se seleccionó este sistema de bases de datos para el presente informe técnico debido a que su mecanismo interno de *Replica Set* ofrece una de las implementaciones más transparentes y comprobables empíricamente para el estudio práctico de la tolerancia a fallos, el particionamiento y los dilemas de consistencia en arquitecturas distribuidas modernas.

2. Arquitectura del Sistema

El pilar de la alta disponibilidad en MongoDB es el conjunto de réplicas (*Replica Set*). Este clúster lógico está compuesto por un Nodo Primario (Primary), responsable exclusivo de canalizar todas las operaciones de escritura, y múltiples Nodos Secundarios (Secondary), cuyo propósito es mantener una copia exacta y en cuasi-tiempo real de los datos mediante un registro circular de operaciones denominado *oplog* [1].

Como se ilustró en el bosquejo inicial trazado a mano (ver fotografía en el Anexo A), la comunicación del sistema tiene dos direcciones claras. Hacia el exterior, fluye desde las aplicaciones cliente conectadas hacia el nodo primario. Hacia el interior, los nodos intercambian señales de control (*heartbeats*) bidireccionales cada dos segundos sobre una red privada para monitorizar la salud mutua y el estado de la topología [2].

3. Análisis de Conceptos de la Unidad 1

3.1. Transparencias de distribución

MongoDB provee un sólido nivel de transparencia de acceso y ubicación. Las aplicaciones cliente se conectan utilizando una única cadena de conexión; es el *driver* el que oculta qué servidor físico contiene el dato, enrutando automáticamente las lecturas. Asimismo, el

sistema destaca en su transparencia de fallos. Si un servidor colapsa, el controlador pone en pausa brevemente las transacciones y las redirige al nuevo líder sin exponer errores, haciendo el fallo invisible a nivel de interfaz [2].

3.2. Teorema CAP y PACELC

Evaluado bajo el Teorema CAP, un Replica Set de MongoDB se clasifica como un sistema CP (Consistencia y Tolerancia a Particiones). Ante una partición de red que aísla al primario, el sistema sacrifica la Disponibilidad: rechaza nuevas escrituras para evitar divergencia de datos hasta que se elija un nuevo líder [1].

Al aplicar el teorema extendido PACELC, su topología se describe como PA/EC. Si hay partición (P), elige consistencia (C). Sin embargo, en estado estable (E), obliga a configurar el balance entre latencia (L) y consistencia (C) utilizando la directiva *Write Concern* [3], [4].

3.3. Modelos de comunicación

Externamente, MongoDB sigue un modelo de cliente-servidor, utilizando conectividad por *sockets* TCP. Hacia el interior del clúster, la replicación de datos rompe con la sincronía: los nodos secundarios extraen (*pull*) las actualizaciones del *oplog* de forma estrictamente asincrónica. Esto evita que el nodo primario quede bloqueado esperando confirmaciones.

3.4. Tolerancia a fallos

El sistema tolera fallos de hardware automatizando la recuperación sin intervención humana usando un algoritmo basado en *heartbeats*. Si un nodo secundario no recibe el "latido"del primario durante 10 segundos, declara la caída del líder y solicita votos. El clúster otorga la victoria al nodo con el *oplog* más actualizado [2].

4. Experimentación Propia

Para comprobar empíricamente la tolerancia a fallos, se ejecutó un experimento local utilizando contenedores. Se desplegó una red virtual aislada (`mongo-cluster`) y se levantaron tres nodos de MongoDB (`mongo1`, `mongo2`, `mongo3`). Tras inicializar el *Replica Set* y confirmar que `mongo1` asumió el rol de *Primary*, se simuló una caída catastrófica deteniendo abruptamente el contenedor principal (`docker stop mongo1`).

Como se ilustra en la Figura 1, la observación del estado del clúster desde `mongo2` demostró que el sistema detectó la falta de *heartbeats*. En un lapso inferior a 10 segundos, los nodos secundarios iniciaron una elección y promovieron a uno de ellos como nuevo líder, evidenciando la capacidad de auto-recuperación del sistema.

5. Ejemplos del Entorno Local

Los dilemas de los sistemas distribuidos son claramente observables en plataformas cotidianas de alta concurrencia utilizadas por la comunidad estudiantil. Por ejemplo, en el portal web del programa Jóvenes en Acción, durante las fechas límite de registro

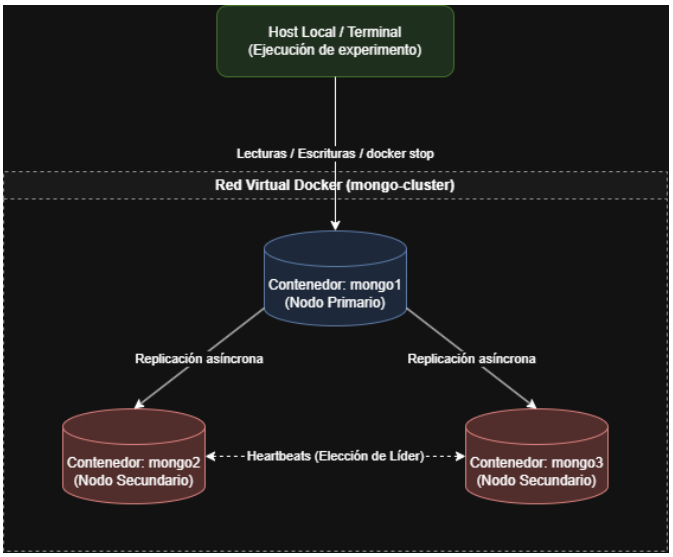


Figura 1: Arquitectura de replicación y tolerancia a fallos del Replica Set de MongoDB desplegado en la red virtual de Docker.

y consulta de asignaciones, la plataforma sufre cuellos de botella severos debido a la avalancha de solicitudes concurrentes. Ante este volumen, el sistema enfrenta dificultades para sincronizar el estado de las postulaciones, viéndose forzado a encolar peticiones o limitar temporalmente el acceso. En una arquitectura como MongoDB, este escenario se mitigaría habilitando el enrutamiento de lecturas hacia los nodos secundarios (*Read Preference: secondary*) o implementando *sharding* para distribuir la carga de escrituras.

Otro caso se observa en la aplicación móvil de Banco Pichincha. Cuando se realiza una transferencia, la plataforma prefiere bloquear temporalmente la operación (sacrificar disponibilidad) si no puede asegurar que el saldo se actualice de manera síncrona (priorizar consistencia). Es un claro ejemplo de un diseño CP en la vida real.

6. Análisis Comparativo

Se establece una comparación con Apache Cassandra, otra base de datos NoSQL fundamentada en principios distintos (Tabla 1) [5], [6].

7. Postura Crítica

MongoDB presenta un *trade-off* técnico evidente: al centralizar escrituras en un único primario, evita conflictos y garantiza consistencia fuerte. Sin embargo, el tiempo de elección de un nuevo líder (10 segundos) implica una ventana de indisponibilidad inaceptable para aplicaciones de ultra-baja latencia. Por ende, es una herramienta especializada para dominios donde la integridad prima sobre la disponibilidad extrema.

8. Conclusiones

La experimentación práctica en contenedores confirma que MongoDB resuelve los desafíos de la distribución priorizando la consistencia sobre la disponibilidad frente a parti-

Cuadro 1: Análisis Arquitectónico de Sistemas Distribuidos: MongoDB vs. Apache Cassandra

Criterio Arquitectónico	**MongoDB**	**Apache Cassandra**
Clasificación Teorema CAP	**CP**: Prioriza Consistencia y Tolerancia a Particiones. Si el nodo primario cae, el sistema suspende escrituras hasta elegir uno nuevo.	**AP**: Prioriza Disponibilidad y Tolerancia a Particiones. Todos los nodos permanecen activos para recibir lecturas y escrituras.
Comportamiento PACELC	**PA/EC**: Ante una partición de red (P), elige Consistencia (C). En estado normal (E), elige Consistencia (C) sobre Latencia (L).	**PA/EL**: Ante una partición de red (P), elige Disponibilidad (A). En estado normal (E), prefiere minimizar la Latencia (L).
Arquitectura de Replicación	**Primario-Secundario (Replica Sets)**: Centraliza las escrituras en un solo nodo. Los secundarios replican los datos del Oplog.	**Peer-to-Peer (Masterless)**: Topología en anillo descentralizada mediante protocolo *Gossip*. Carece de un punto único de fallo.
Transparencia de Fallos	**Por proceso de elección**: Los nodos secundarios detectan la desconexión y ejecutan una votación para promover al nuevo primario.	**Intrínseca por enrutamiento**: Si un nodo falla, el coordinador redirige automáticamente la petición a las siguientes réplicas del anillo.
Manejo de Concurrencia	Bloqueos granulares a nivel de documento y control de concurrencia optimista gestionado por el motor *WiredTiger*.	Resolución de conflictos distribuida mediante marcas de tiempo de cliente (*Last Write Wins*).
Estrategia de Escalabilidad	**Sharding**: Escalamiento horizontal que exige desplegar enrutadores (*mongos*) y servidores de configuración.	**Lineal Nativa**: Escalamiento horizontal fluido; se añaden nodos al clúster sin necesidad de reconfigurar una arquitectura central.

ciones de red. Su modelo de *Replica Set* logra ocultar eficazmente los fallos de hardware al cliente mediante procesos electorales automatizados, validando las restricciones ineludibles planteadas por el teorema PACELC.

9. Declaración de uso de Inteligencia Artificial

Se declara el uso del modelo de lenguaje Gemini 3.8 estrictamente como asistente de soporte sintáctico para la corrección de comandos mal escritos en la terminal durante la ejecución de Docker, y como corrector ortotipográfico en latex (revisión de faltas ortográficas, validación de signos de puntuación y uso adecuado de comas) por parte de la documentación.

Referencias

[1] W. Schultz y M. Demirbas, "Design and Modular Verification of Distributed Transactions in MongoDB," *Proceedings of the VLDB Endowment*, vol. 18, n.º 12, págs. 5045-5058, 2025, doi: 10.14778/3750601.3750626.

[2] MongoDB, Inc. "Replication - MongoDB Manual," visitado 13 de nov. de 2026. dirección: https://www.mongodb.com/docs/manual/replication/

[3] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, "Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications," *Journal of Cloud Computing*, vol. 13, pág. 158, 2024, doi: 10.1186/s13677-024-00716-7.

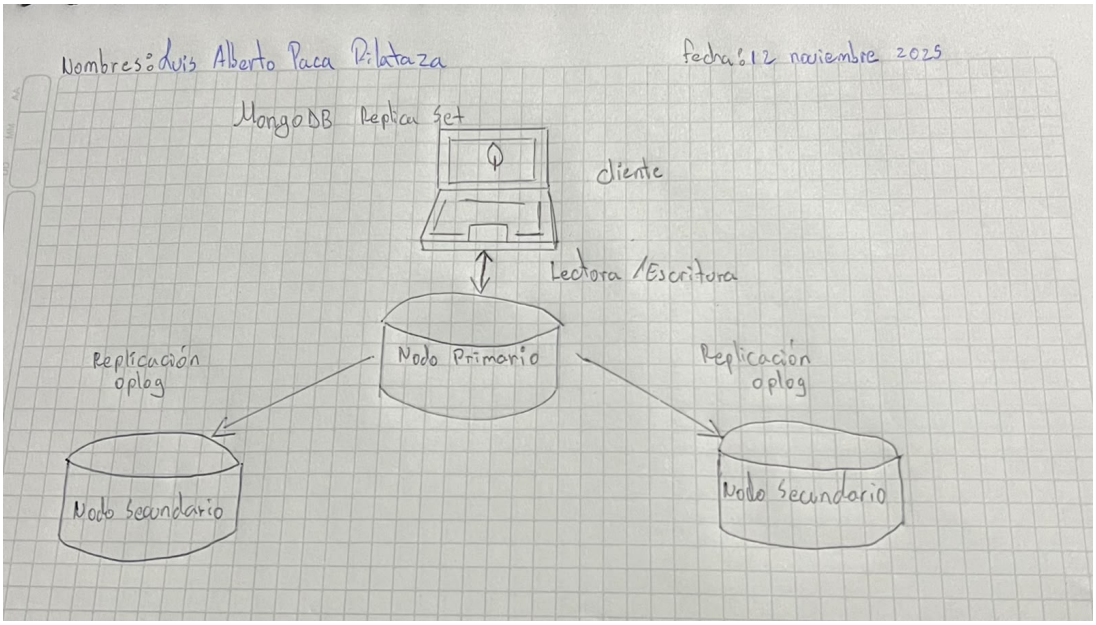
[4] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, "Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB," *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025, doi: 10.1109/ACCESS.2025.3558923.

[5] C. A. Győrödi, D. V. Dumşe-Burescu, D. R. Zmaranda y R. Ş. Győrödi, "A Comparative Study of MongoDB and Document-Based MySQL for Big Data Application Data Management," *Big Data and Cognitive Computing*, vol. 6, n.º 2, pág. 49, 2022, doi: 10.3390/bdcc6020049.

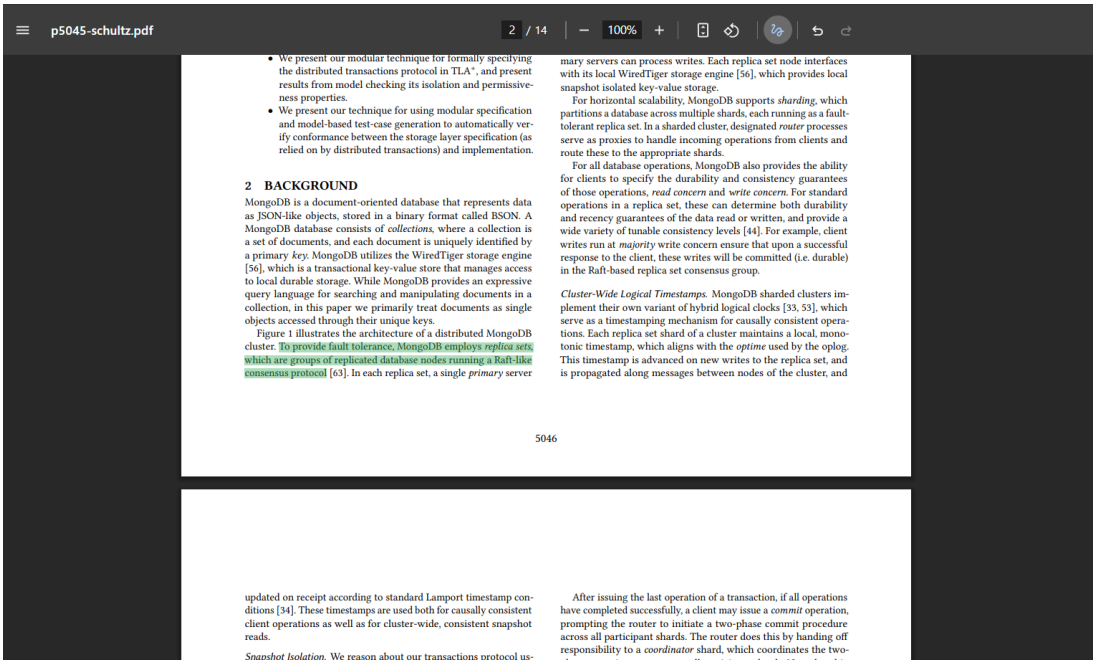
[6] S. A. Dubey y A. Saxena, "Performance Analysis of NoSQL Databases for Healthcare Applications: A Benchmarking Study of MongoDB, Cassandra and Couchbase," *Journal of Information Systems Engineering and Management*, vol. 10, n.º 25s, págs. 380-398, 2025, doi: 10.52783/jisem.v10i25s.4030.

10. Anexos

A. Esquema Inicial a Mano



B. Capturas de Lectura de Fuentes



- We present our modular technique for formally specifying the distributed transactions protocol in TLA+, and present results from model checking its isolation and permissiveness properties.
- We present our technique for using modular specification and model-based test-case generation to automatically verify conformance between the storage layer specification (as relied on by distributed transactions) and implementation.

2 BACKGROUND

MongoDB is a document-oriented database that represents data as JSON-like objects, stored in a binary format called BSON. A MongoDB database consists of *collections*, where a collection is a set of documents, and each document is uniquely identified by a primary *key*. MongoDB utilizes the WiredTiger storage engine [56], which is a transactional key-value store that manages access to local durable storage. While MongoDB provides an expressive query language for searching and manipulating documents in a collection, in this paper we primarily treat documents as single objects accessed through their unique keys.

Figure 1 illustrates the architecture of a distributed MongoDB cluster. To provide fault tolerance, MongoDB employs *replica sets*, which are groups of replicated database nodes running a Raft-like consensus protocol [63]. In each replica set, a single *primary* server

mary servers can process writes. Each replica set node interfaces with its local WiredTiger storage engine [56], which provides local snapshot isolated key-value storage.

For horizontal scalability, MongoDB supports *sharding*, which partitions a database across multiple shards, each running as a fault-tolerant replica set. In a sharded cluster, designated *router* processes serve as proxies to handle incoming operations from clients and route these to the appropriate shards.

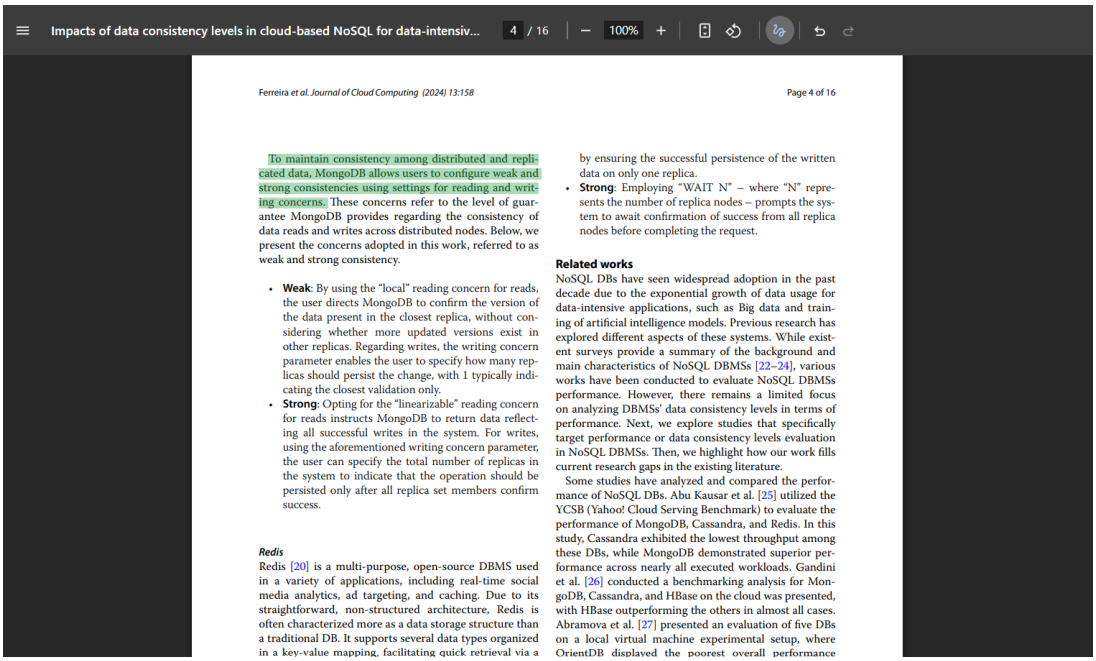
For all database operations, MongoDB also provides the ability for clients to specify the durability and consistency guarantees of those operations, *read concern* and *write concern*. For standard operations in a replica set, these can determine both durability and recency guarantees of the data read or written, and provide a wide variety of tunable consistency levels [44]. For example, client writes run at *majority* write concern ensure that upon a successful response to the client, these writes will be committed (i.e. durable) in the Raft-based replica set consensus group.

Cluster-Wide Logical Timestamps. MongoDB sharded clusters implement their own variant of hybrid logical clocks [33, 53], which serve as a timestamping mechanism for causally consistent operations. Each replica set shard of a cluster maintains a local, monotonic timestamp, which aligns with the *optime* used by the oplog. This timestamp is advanced on new writes to the replica set, and is propagated along messages between nodes of the cluster, and

updated on receipt according to standard Lamport timestamp conditions [34]. These timestamps are used both for causally consistent client operations as well as for cluster-wide, consistent snapshot reads.

Snapshot Isolation. We reason about our transactions protocol us-

After issuing the last operation of a transaction, if all operations have completed successfully, a client may issue a *commit* operation, prompting the router to initiate a two-phase commit procedure across all participant shards. The router does this by handing off responsibility to a *coordinator* shard, which coordinates the two-



To maintain consistency among distributed and replicated data, MongoDB allows users to configure weak and strong consistencies using settings for reading and writing concerns. These concerns refer to the level of guarantee MongoDB provides regarding the consistency of data reads and writes across distributed nodes. Below, we present the concerns adopted in this work, referred to as weak and strong consistency.

- **Weak**: By using the "local" reading concern for reads, the user directs MongoDB to confirm the version of the data present in the closest replica, without considering whether more updated versions exist in other replicas. Regarding writes, the writing concern parameter enables the user to specify how many replicas should persist the change, with 1 typically indicating the closest validation only.
- **Strong**: Opting for the "linearizable" reading concern for reads instructs MongoDB to return data reflecting all successful writes in the system. For writes, using the aforementioned writing concern parameter, the user can specify the total number of replicas in the system to indicate that the operation should be persisted only after all replica set members confirm success.

Redis

Redis [20] is a multi-purpose, open-source DBMS used in a variety of applications, including real-time social media analytics, ad targeting, and caching. Due to its straightforward, non-structured architecture, Redis is often characterized more as a data storage structure than a traditional DB. It supports several data types organized in a key-value mapping, facilitating quick retrieval via a

by ensuring the successful persistence of the written data on only one replica.

- **Strong**: Employing "WAIT N" – where "N" represents the number of replica nodes – prompts the system to await confirmation of success from all replica nodes before completing the request.

Related works

NoSQL DBs have seen widespread adoption in the past decade due to the exponential growth of data usage for data-intensive applications, such as Big data and training of artificial intelligence models. Previous research has explored different aspects of these systems. While existent surveys provide a summary of the background and main characteristics of NoSQL DBMSs [22–24], various works have been conducted to evaluate NoSQL DBMSs performance. However, there remains a limited focus on analyzing DBMSs' data consistency levels in terms of performance. Next, we explore studies that specifically target performance or data consistency levels evaluation in NoSQL DBMSs. Then, we highlight how our work fills current research gaps in the existing literature.

Some studies have analyzed and compared the performance of NoSQL DBs. Abu Kausar et al. [25] utilized the YCSB (Yahoo! Cloud Serving Benchmark) to evaluate the performance of MongoDB, Cassandra, and Redis. In this study, Cassandra exhibited the lowest throughput among these DBs, while MongoDB demonstrated superior performance across nearly all executed workloads. Gandini et al. [26] conducted a benchmarking analysis for MongoDB, Cassandra, and HBase on the cloud was presented, with HBase outperforming the others in almost all cases. Abramova et al. [27] presented an evaluation of five DBs on a local virtual machine experimental setup, where OrientDB displayed the poorest overall performance

Table 1. Comparison of the characteristics between MongoDB and document-based MySQL.

	MongoDB	Document-Based MySQL
Data model	BSON objects [8]—key–value documents, each document identified via a unique identifier, offer predefined methods for all relational MySQL commands.	BSON objects [8]—key–value documents, each document identified via a unique identifier, does not require defining a fixed structure during document creation [13,14].
Query model	Queries are expressed as JSON objects and are sent by the database driver using the "find" method [15]; more complex queries could be expressed using a Map Reduce operation for batch processing of data and aggregation operations [8]. The user specifies the map and reduces functions in JavaScript, and they are executed on the server side [15]. The results of the operation are possible to store in a temporary collection that is automatically removed after the client receives the results, or they can be stored in a permanent collection, so that they are always available. MongoDB has introduced the $lookup operator in version 3.2. that performs Left Outer Join (called left join) operations with two or more collections [9].	Document-based MySQL allows developers, through the X Dev API, to work with relational tables and JSON document collections [13]. The X DEV API provides an easy-to-use API for performing CRUD operations, being optimized and extensible for performing these operations. In the case of more complex operations, knowledge of relational MySQL is very helpful for writing queries in document-based MySQL, because certain search or edit methods take as a parameter part of the query from relational MySQL; more precisely, the conditions, the part after the "where" clause.
Replication model	Provides Master–Slave replication and replica sets, where data is asynchronously replicated between servers [8]. A replica set contains several data-bearing nodes and optionally, one arbiter node. Of the data-bearing nodes, one and only one member is deemed the primary node, while the other nodes are deemed secondary nodes [15]. Replica sets provide a high performance for replication with automated failure handling, while sharded clusters make it possible to divide large data sets over different machines that are transparent to the users [13]. MongoDB users combine replica sets and sharded clusters to provide high levels of redundancy in data sets that are transparent for applications [8].	Provides multi-document transaction support and full ACID compliance for schema-less JSON documents having a high availability by using all the advantages of MySQL Group Replication and the InnoDB Cluster to scale-out applications [13,14]. Documents are replicated across all members of the high-availability group, and transactions are committed in sync across masters. Any source server can take over from another source server if one fails, without downtime [13].

The structure of both databases is especially suitable for flexible applications whose structure is not static from the beginning, and it is expected that there will be many changes along the way. When it comes to large volumes of data—in the order of millions, even if thousands of queries per second are allowed in any type of database, the way in which they manage operations and the optimizations that come with the package define their efficiency, both being optimized to operate upon a large volume of data. However, in MongoDB, access is based on the roles defined for each user, and in document-based MySQL, access is